

How Computers Work · Section 11

A First Look at C and Memory

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. Why Security People Learn a Little C 1

2. A Tiny C Program 2

3. Variables Are Places in Memory 3

4. Pointers, Gently 3

5. The Stack and a Buffer 4

6. What a Buffer Overflow Is (a Preview) 5

7. Memory Safety, and Why It Matters 6

Common Mistakes 6

Summary 7

Check Your Understanding 7

Practice 8

What You Will Learn

- Why security people learn a little C.
- A tiny C program, read from zero.
- How variables are really places in memory, with addresses.
- What a pointer is, gently.
- The stack and a buffer.
- What a buffer overflow is, as a preview.
- Memory safety, and why it matters so much in security.

Before You Start

Read lessons 2, 6, and 8. You know bytes and addresses, the stack in a process's memory, and that C is compiled to machine code. This lesson gets hands-on with memory using the smallest possible amount of **C**. The goal is not to teach you C (that is a full course), but to let you *see* memory directly, because this is the foundation the Binary Exploitation course (18) is built on. Every program here is complete, compiled with Clang, and shown with its real output. One address will appear; note that **memory addresses change every time a program runs** (a defense called ASLR, which you will meet later), so yours will differ.

---

1. Why Security People Learn a Little C

C is an old language, but it remains central to security for a specific reason: **C works directly with memory.** Most modern languages (Python, JavaScript, Java) deliberately hide memory from you, they manage it automatically and stop you from touching raw addresses, for safety and convenience. C does the opposite: it lets you see and manipulate memory directly, addresses, bytes, the stack, the heap.

That makes C both powerful and dangerous, and that is exactly why it matters here:

- **The systems you will attack and defend are often written in C (or C++).** Operating system kernels, many core programs, and much embedded and IoT software are C. Understanding them means understanding C's view of memory.
- **The most serious class of vulnerabilities, memory-corruption bugs, lives in C-like languages.** Buffer overflows, use-after-free, and their relatives (the whole Binary

Exploitation course) happen because C lets a program misuse memory. You cannot understand these attacks without seeing how C touches memory.

- **It makes memory concrete.** The stack, addresses, and pointers from earlier lessons stop being abstract once you see them in a few lines of C.

You will not become a C programmer in this lesson, and you do not need to. You need to read a handful of tiny programs and *watch what happens to memory*. That is enough to demystify the attacks later.

2. A Tiny C Program

Here is a complete C program. Read it slowly; the comments explain each part.

```
#include <stdio.h>

int main(void) {
    int price = 2500;
    printf("price is %d\n", price);
    printf("price lives at address %p\n", (void *)&price);
    return 0;
}
```

What each piece means:

- `#include <stdio.h>` brings in the standard library so we can use `printf` (lesson 8's libraries).
- `int main(void)` is the **main function**, where the program starts (like `main` in the lesson-1 idea of a starting point).
- `int price = 2500;` creates a **variable** named `price`, of type `int` (a whole number), and puts 2500 in it.
- `printf(...)` prints. `%d` is replaced by the number; `%p` is replaced by an **address** (topic 3).
- `&price` means "the address *of* `price`", the `&` is the key new idea.

Running it:

```
$ clang -o variables variables.c
$ ./variables
price is 2500
price lives at address 0x16b376408
```

The first line is the value, 2500. The second line is something new and important: the **address** where that value lives in memory, shown in hex (lesson 2). Your address will be different, and will change each run. That second line is the whole point of this lesson: a variable is not just a value, it is a value *stored at a place*, and that place has an address.

3. Variables Are Places in Memory

The program just showed it directly, but the idea deserves its own moment, because everything about memory attacks depends on it.

A variable is a named place in memory that holds a value. When you write `int price = 2500;`, the computer:

1. Sets aside a small region of memory (for an `int`, usually 4 bytes) somewhere, often on the **stack** (lesson 6), since `price` is a local variable inside a function.
2. Gives that region the name `price`, so you can refer to it.
3. Stores the bytes for 2500 there.

So a variable has three things: a **name** (`price`), a **value** (2500), and an **address** (where it lives, like `0x16b376408`). Normally you only think about the name and the value, the language hides the address. C lets you ask for the address with `&`, which is why we could print it.

Why does this matter for security? Because **attacks on memory are about addresses and the bytes at them, not names.** An attacker does not care that a variable is called `price`; they care *where* it is in memory and what is next to it. When you read about an exploit "overwriting" a value or "controlling" an address, this is the world it lives in: named values are really just bytes at addresses, and whoever controls the bytes at the right address controls the program. This lesson makes that concrete; the exploitation course weaponizes it.

4. Pointers, Gently

If a variable has an address, you can store that address in another variable. A variable that holds an address is called a **pointer**. Pointers are famous for confusing beginners, so we take them gently, with a program:

```c
#include <stdio.h>

int main(void) {
    int price = 2500;
    int *p = &price;       // p holds the address of price
    printf("value through pointer: %d\n", *p);
    *p = 3000;             // change price via the pointer
    printf("price is now: %d\n", price);
    return 0;
}
```

Two new bits of notation, and one idea:

- `int *p = &price;` creates a pointer `p` and stores in it the *address of* `price` (the `&` from topic 2). So `p` does not hold 2500; it holds *where 2500 lives*.

- `*p` means "the value *at* the address `p` holds", following the pointer to what it points at. Reading `*p` gives 2500.
- `*p = 3000;` writes 3000 *to the address `p` points at*, which is `price`'s location, so it changes `price` itself.

Running it:

```
$ ./pointers
value through pointer: 2500
price is now: 3000
```

We changed `price` to 3000 *without ever naming `price`* on that line, only by going through its address. That is the essence of a pointer: **a pointer is a way to read and write memory by address.** Think of it like a house address written on a slip of paper: the slip is not the house, but anyone holding the slip can go to the house and change what is inside. Pointers are what make C powerful, and they are also where much goes wrong: a pointer holding a *wrong* address lets a program read or write memory it should not. That is the doorway to the next two topics.

5. The Stack and a Buffer

Recall the stack (lesson 6): the region of memory where functions keep their local variables, growing and shrinking as functions are called and return. Our `price` variable lived on the stack. Now meet a **buffer**: a block of memory set aside to hold a sequence of bytes, such as text. Here is a buffer on the stack, used safely:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[8];                 // room for 7 letters + end marker
    strncpy(name, "Dara", sizeof(name) - 1);
    name[sizeof(name) - 1] = '\0';
    printf("hello, %s\n", name);
    return 0;
}
```

- `char name[8];` sets aside a buffer of 8 bytes on the stack, room for 7 characters of text plus the special end-marker byte (`'\0'`, the "null terminator") that C uses to mark where text ends.
- `strncpy(name, "Dara", sizeof(name) - 1);` copies the text `"Dara"` into the buffer, but **at most 7 bytes**, deliberately not more than the buffer can hold.

Running it:

```
$ ./buffer
hello, Dara
```

The key word above is **at most 7 bytes**. The buffer is a fixed size, 8 bytes, but the data you copy into it might be any length. A careful program always checks that what it writes *fits*. The whole drama of the next topic is what happens when a program does *not* check, and writes more data than the buffer can hold.

6. What a Buffer Overflow Is (a Preview)

Here is the single most famous class of memory bug, and you now have everything needed to understand it. A **buffer overflow** happens when a program writes **more data into a buffer than the buffer can hold**, so the extra data spills past the end of the buffer and over whatever sits next to it in memory.

Imagine the unsafe version of the previous program, which copies text into the 8-byte buffer *without* limiting the length (shown here only to illustrate the bug, it is exactly what you must never write):

```
char name[8];
strcpy(name, user_input);   // DANGER: no length check!
```

If `user_input` is `"Dara"`, fine, it fits. But if it is 50 characters long, `strcpy` happily writes all 50 bytes, running 42 bytes *past* the end of the 8-byte buffer, overwriting whatever is next on the stack. And recall from lesson 6 what sits near local variables on the stack: the **return address**, the record of where the CPU should go when the function finishes.

This is the attack. By supplying carefully crafted over-long input, an attacker can overflow the buffer and **overwrite the return address** with an address of their choosing. When the function returns, the CPU jumps there, running instructions the attacker picked instead of the real program. Over-long *data* has become *control* of the program. This is the classic stack buffer overflow, and it has been behind countless real-world compromises.

You are not exploiting anything here, that is the Binary Exploitation course (18), done in a lab, on your own targets, under the ethics rule. The point of this preview is that the attack is no longer mysterious: it is a direct consequence of everything you have learned, buffers are fixed-size, the stack holds the return address nearby, C does not stop you writing past a buffer, and whoever controls the bytes at the right address controls the program. Defenses you will meet, like **NX** (lesson 6) and **ASLR** (the changing addresses in this lesson), exist specifically to make this attack harder.

7. Memory Safety, and Why It Matters

Step back and ask: why do buffer overflows exist at all? Because C (and C++) are **not memory-safe**: they let a program read and write memory out of bounds, use a pointer to the wrong place, or use memory after it is freed, without stopping it. The program just does it, and the result is a bug or an exploit. This single property is responsible for a large share of the most serious security vulnerabilities in history.

The response, across the whole industry, comes in two forms, and both matter to you:

- **Defenses that make exploitation harder** on existing C/C++ code: NX (non-executable stack), ASLR (randomized addresses), stack canaries, and more. You saw NX and ASLR named already; Binary Exploitation studies them and the techniques that try to defeat them.
- **Memory-safe languages that prevent the bugs entirely.** Languages like **Rust**, and managed languages like Python, Java, and Go, do not let a program write past a buffer or misuse a pointer, they check, or manage memory for you. This is why there is a strong modern push to write new security-sensitive software in memory-safe languages: it removes an entire class of vulnerabilities at the source.

So the humble buffer overflow connects to huge themes: why some languages are ”safer” than others, why old C code is a perennial risk, and why the industry is slowly moving to memory-safe languages. Understanding memory, which you now do, is what lets you see all of this clearly.

You have now looked at memory up close and seen, in a few lines, where the most serious attacks come from. One lesson remains: putting everything from this whole course together on a real, running system, inspecting its processes, files, and bytes with your own hands.

Common Mistakes

1. Thinking a variable is only a value. It is a value stored at an address. Memory attacks care about the address and the bytes, not the name.

2. Being scared of pointers. A pointer is just a variable holding an address, a slip of paper with a house address on it. It lets you read/write memory by address.

3. Assuming data always fits its buffer. A buffer is a fixed size; input can be any length. Not checking the length is the bug behind buffer overflows.

4. Thinking a buffer overflow is just a crash. Overwriting the return address can hand control of the program to an attacker. Over-long data becomes control.

5. Believing all languages are equally safe. C/C++ are not memory-safe and allow these bugs; memory-safe languages (Rust, Python, Java) prevent them. The choice matters.

Summary

Term	Remember
Why C	It works directly with memory; the systems and bugs that matter most are C/C++
Variable	A named place in memory: a name, a value, and an **address** (`&` gets the address)
Pointer	A variable that holds an address; `*p` reads/writes the memory it points at
Buffer	A fixed-size block of memory for a sequence of bytes
Buffer overflow	Writing past a buffer's end, over nearby memory, can overwrite the return address
From data to control	Crafted over-long input can redirect the CPU to the attacker's code
Memory safety	C/C++ allow these bugs; memory-safe languages (Rust, Python, ...) prevent them

Check Your Understanding

1. Why do security people learn a little C, rather than only "safe" languages?
2. What three things does a variable have, and which does C let you access with `&`?
3. In plain words, what is a pointer, and what does `*p` mean?
4. What is a buffer, and why must a program check the length of what it writes into one?
5. In a buffer overflow, what spills where, and which nearby value is the dangerous target?
6. How does an attacker turn an over-long input into control of a program?
7. What does "memory-safe" mean, and name one memory-safe and one memory-unsafe language.

Answers

1. Because the systems they attack and defend (OS kernels, core and IoT software) are often written in C/C++, and the most serious vulnerabilities (buffer overflows and relatives) come from C's direct, unchecked access to memory. You cannot understand those attacks without understanding C's view of memory.
2. A name, a value, and an address. C lets you access the address with `&` (e.g. `&price`).
3. A pointer is a variable that holds an address, where another value lives. `*p` means "the value at the address `p` holds", so it reads (or, when assigned, writes) the memory `p` points

at.

4. A buffer is a fixed-size block of memory for a sequence of bytes. The data copied in can be any length, so the program must check it fits; not checking lets data spill past the buffer (a buffer overflow).
5. The extra data spills past the end of the buffer and over whatever is next to it in memory. The dangerous target is the return address on the stack.
6. By supplying crafted over-long input that overflows the buffer and overwrites the return address with an address they chose, so when the function returns, the CPU jumps to the attacker's instructions.
7. Memory-safe means the language prevents out-of-bounds access and pointer misuse (it checks or manages memory), so these bugs can't happen. Memory-safe: Rust (or Python/Java/Go). Memory-unsafe: C (or C++).

Practice

1. Explain, using the "slip of paper with a house address" analogy, what a pointer is and how `*p = 3000` changes a variable without naming it.
2. A program has `char code[4];` and copies a user-supplied coupon code into it with no length check. Explain what goes wrong if the user enters a 20-character code, and why it is dangerous, not just a bug.
3. If you have a C compiler, compile and run the `variables` program from topic 2 twice. Does the printed address stay the same or change between runs? What defense does that changing address represent?

A Worked Example

1. "A pointer is like a slip of paper with a house's address written on it. The slip isn't the house, but anyone holding it can go to that house and change what's inside. So `p` holds `price`'s address, and `*p = 3000` means 'go to the address on the slip and put 3000 there', which changes `price` itself, without writing the name `price` on that line."
2. The buffer `code` holds only 4 bytes, but 20 characters are copied in, so 16 bytes spill past the end of the buffer onto the stack, over whatever is next, including, eventually, the return address. It's dangerous rather than just a crash because an attacker who controls that input can overwrite the return address and redirect the program to their own code.
3. The address changes between runs (for example `0x16b376408` one time, a different value the next). That changing address is ASLR (Address Space Layout Randomization), a defense that randomizes where things sit in memory so attackers cannot rely on fixed addresses.

Course mark icon from Material Symbols by Google, Apache License 2.0.